

Neural Network Regression

Kachun Huang

2025-03-06

Neural Network Regression

library Import

```
library(neuralnet)
```

```
## Warning: package 'neuralnet' was built under R version 4.3.3
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr    1.5.1
## v ggplot2    3.5.1      v tibble     3.2.1
## v lubridate  1.9.3      v tidyr      1.3.0
## v purrr      1.0.2
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::compute() masks neuralnet::compute()
```

```
## x dplyr::filter()  masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.3.3
```

```
## Loading required package: lattice
```

```
##
```

```
## Attaching package: 'caret'
```

```
##
```

```
## The following object is masked from 'package:purrr':
```

```
##
```

```
## lift
```

```
library(MASS)
```

```
##
## Attaching package: 'MASS'
##
## The following object is masked from 'package:dplyr':
##
##      select
```

Data Import

```
data("Boston")
house = Boston
head(house)
```

```
##      crim zn indus chas   nox   rm  age   dis rad tax ptratio  black lstat
## 1 0.00632 18  2.31    0 0.538 6.575 65.2 4.0900   1 296    15.3 396.90  4.98
## 2 0.02731  0  7.07    0 0.469 6.421 78.9 4.9671   2 242    17.8 396.90  9.14
## 3 0.02729  0  7.07    0 0.469 7.185 61.1 4.9671   2 242    17.8 392.83  4.03
## 4 0.03237  0  2.18    0 0.458 6.998 45.8 6.0622   3 222    18.7 394.63  2.94
## 5 0.06905  0  2.18    0 0.458 7.147 54.2 6.0622   3 222    18.7 396.90  5.33
## 6 0.02985  0  2.18    0 0.458 6.430 58.7 6.0622   3 222    18.7 394.12  5.21
##   medv
## 1 24.0
## 2 21.6
## 3 34.7
## 4 33.4
## 5 36.2
## 6 28.7
```

```
str(house)
```

```
## 'data.frame':   506 obs. of  14 variables:
##  $ crim   : num  0.00632 0.02731 0.02729 0.03237 0.06905 ...
##  $ zn     : num  18 0 0 0 0 0 12.5 12.5 12.5 12.5 ...
##  $ indus  : num  2.31 7.07 7.07 2.18 2.18 2.18 7.87 7.87 7.87 7.87 ...
##  $ chas   : int   0 0 0 0 0 0 0 0 0 0 ...
##  $ nox    : num  0.538 0.469 0.469 0.458 0.458 0.458 0.524 0.524 0.524 0.524 ...
##  $ rm     : num  6.58 6.42 7.18 7 7.15 ...
##  $ age    : num  65.2 78.9 61.1 45.8 54.2 58.7 66.6 96.1 100 85.9 ...
##  $ dis    : num  4.09 4.97 4.97 6.06 6.06 ...
##  $ rad    : int   1 2 2 3 3 3 5 5 5 5 ...
##  $ tax    : num  296 242 242 222 222 222 311 311 311 311 ...
##  $ ptratio: num  15.3 17.8 17.8 18.7 18.7 18.7 15.2 15.2 15.2 15.2 ...
##  $ black  : num  397 397 393 395 397 ...
##  $ lstat  : num  4.98 9.14 4.03 2.94 5.33 ...
##  $ medv   : num  24 21.6 34.7 33.4 36.2 28.7 22.9 27.1 16.5 18.9 ...
```

The Boston housing price data we will use for our homework is taken from the Kaggle competition site (<https://www.kaggle.com/vikrishnan/boston-house-prices>..) We will treat this dataset as our entire data.

Each record in the database describes a Boston suburb or town. The data was drawn

from the Boston Standard Metropolitan Statistical Area (SMSA) in 1970. Our data has 506 observations and 14 variables:

Attribute Information (in order):

- CRIM: per capita crime rate by town
- ZN: proportion of residential land zoned for lots over 25,000 sq.ft.
- INDUS: proportion of non-retail business acres per town
- CHAS: Charles River dummy variable (= 1 if tract bounds river; 0 otherwise)
- NOX: nitric oxides concentration (parts per 10 million)
- RM: average number of rooms per dwelling
- AGE: proportion of owner-occupied units built before 1940
- DIS: weighted distances to five Boston employment centers
- RAD: index of accessibility to radial highways
- TAX: full-value property-tax rate per 10,000 dollars
- PTRATIO: pupil-teacher ratio by town
- Black: The result of the equation $B = 1000(B_k - 0.63)^2$ where B_k is the proportion of blacks by town
- LSTAT: %lower status of the population
- MEDV: Median value of owner-occupied homes in 1000 dollars

Data Preprocessing

Exclude RAD Columns

```
house <- subset(house, select = -c(rad))
head(house)
```

```
##      crim zn  indus chas   nox    rm  age    dis tax ptratio  black lstat medv
## 1 0.00632 18  2.31    0 0.538 6.575 65.2 4.0900 296    15.3 396.90  4.98 24.0
## 2 0.02731  0  7.07    0 0.469 6.421 78.9 4.9671 242    17.8 396.90  9.14 21.6
## 3 0.02729  0  7.07    0 0.469 7.185 61.1 4.9671 242    17.8 392.83  4.03 34.7
## 4 0.03237  0  2.18    0 0.458 6.998 45.8 6.0622 222    18.7 394.63  2.94 33.4
## 5 0.06905  0  2.18    0 0.458 7.147 54.2 6.0622 222    18.7 396.90  5.33 36.2
## 6 0.02985  0  2.18    0 0.458 6.430 58.7 6.0622 222    18.7 394.12  5.21 28.7
```

```
str(house)
```

```
## 'data.frame': 506 obs. of 13 variables:
## $ crim : num 0.00632 0.02731 0.02729 0.03237 0.06905 ...
## $ zn : num 18 0 0 0 0 12.5 12.5 12.5 12.5 ...
## $ indus : num 2.31 7.07 7.07 2.18 2.18 2.18 7.87 7.87 7.87 7.87 ...
## $ chas : int 0 0 0 0 0 0 0 0 0 0 ...
## $ nox : num 0.538 0.469 0.469 0.458 0.458 0.458 0.524 0.524 0.524 0.524 ...
## $ rm : num 6.58 6.42 7.18 7 7.15 ...
## $ age : num 65.2 78.9 61.1 45.8 54.2 58.7 66.6 96.1 100 85.9 ...
## $ dis : num 4.09 4.97 4.97 6.06 6.06 ...
## $ tax : num 296 242 242 222 222 222 311 311 311 311 ...
## $ ptratio: num 15.3 17.8 17.8 18.7 18.7 18.7 15.2 15.2 15.2 15.2 ...
## $ black : num 397 397 393 395 397 ...
## $ lstat : num 4.98 9.14 4.03 2.94 5.33 ...
## $ medv : num 24 21.6 34.7 33.4 36.2 28.7 22.9 27.1 16.5 18.9 ...
```

Normalize Data

```
house <- data.frame(scale(house))
head(house)
```

```
##      crim      zn      indus      chas      nox      rm      age
## 1 -0.4193669  0.2845483 -1.2866362 -0.2723291 -0.1440749  0.4132629 -0.1198948
## 2 -0.4169267 -0.4872402 -0.5927944 -0.2723291 -0.7395304  0.1940824  0.3668034
## 3 -0.4169290 -0.4872402 -0.5927944 -0.2723291 -0.7395304  1.2814456 -0.2655490
## 4 -0.4163384 -0.4872402 -1.3055857 -0.2723291 -0.8344581  1.0152978 -0.8090878
## 5 -0.4120741 -0.4872402 -1.3055857 -0.2723291 -0.8344581  1.2273620 -0.5106743
## 6 -0.4166314 -0.4872402 -1.3055857 -0.2723291 -0.8344581  0.2068916 -0.3508100
##      dis      tax      ptratio      black      lstat      medv
## 1  0.140075 -0.6659492 -1.4575580  0.4406159 -1.0744990  0.1595278
## 2  0.556609 -0.9863534 -0.3027945  0.4406159 -0.4919525 -0.1014239
## 3  0.556609 -0.9863534 -0.3027945  0.3960351 -1.2075324  1.3229375
## 4  1.076671 -1.1050216  0.1129203  0.4157514 -1.3601708  1.1815886
## 5  1.076671 -1.1050216  0.1129203  0.4406159 -1.0254866  1.4860323
## 6  1.076671 -1.1050216  0.1129203  0.4101651 -1.0422909  0.6705582
```

Data Splitting

```
set.seed(123)
training.sample <- house$medv %>% createDataPartition(p=0.75, list=FALSE)
train.data <- house[training.sample,]
test.data <- house[-training.sample,]
```

In your own research, you can follow this:

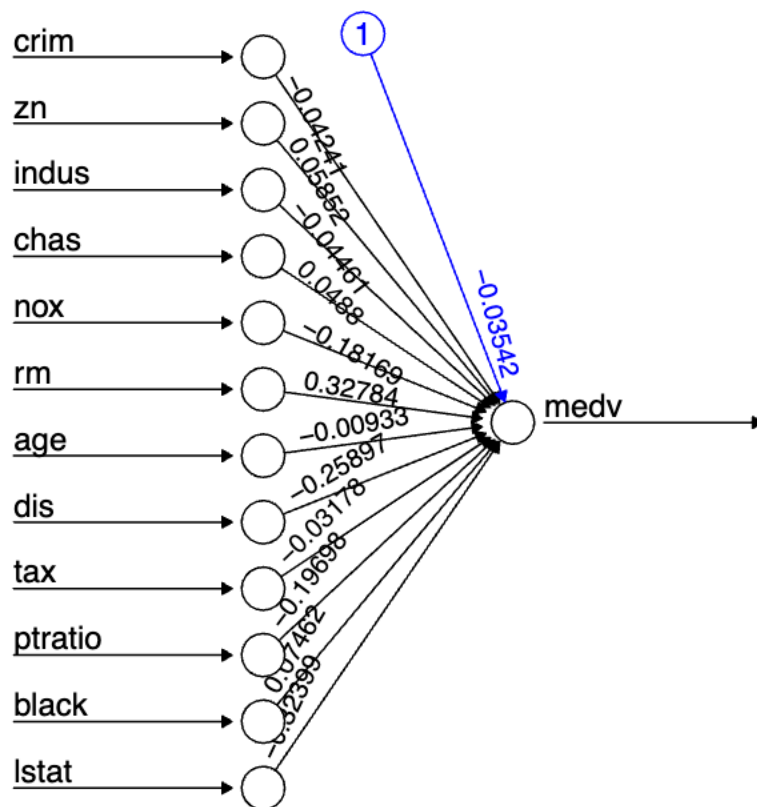
- 1) Always scale continuous features before fitting a neural network.
- 2) Categorical variables (one-hot encoded) do not need scaling.

Neural Network with no hidden layer

```
nn_no_model <- neuralnet(  
  medv ~.,  
  train.data,  
  hidden = 0,  
  err.fct = "sse",  
  linear.output = TRUE  
)
```

Plot

```
plot(nn_no_model, rep="best")
```



Predict on Test Data

```
#predict on test data  
pred_nn_no <- predict(nn_no_model, test.data)
```

Descal

```
#compute mean and sd
mean <- mean(house$medv)
sd <- sd(house$medv)

# compute y(unscaled)
y <- test.data$medv*sd+mean

#compute y^hat (unscaled)
yhat <- pred_nn_no*sd+mean
```

Compute Test MSE

```
(MSE.nn.1 = RMSE(y,yhat)^2)
```

```
## [1] 0.5072648
```

or

```
sum((y-yhat)^2)/length(y)
```

```
## [1] 0.5072648
```

Neural Network with 1 hidden layer

```
nn_one_model <- neuralnet(
  medv ~.,
  train.data,
  hidden = 3,
  err.fct = "sse",
  linear.output = TRUE
)
```

Plot

```
plot(nn_one_model, rep="best")
```

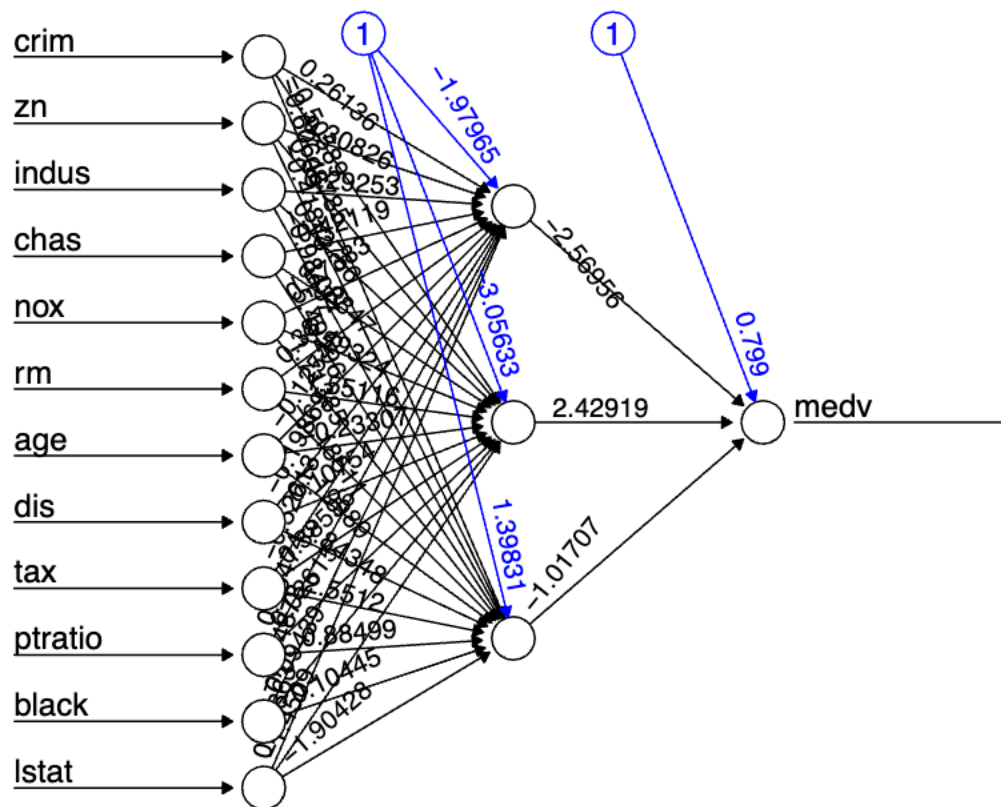


Figure 10.333544: Output: 3333

Predict on Test Data

```
#predict on test data
pred_nn_one <- predict(nn_one_model, test.data)
```

Descale

```
#compute mean and sd
mean <- mean(house$medv)
sd <- sd(house$medv)

# compute y(unscaled)
y <- test.data$medv*sd+mean

#compute y^hat (unscaled)
yhat <- pred_nn_one*sd+mean
```

Compute Test MSE

```
(MSE.nn.2 = RMSE(y,yhat)^2)
```

```
## [1] 0.3465452
```

or

```
sum((y-yhat)^2)/length(y)
```

```
## [1] 0.3465452
```

Multiple Linear Regression

```
mlr <- lm(medv~., data=train.data)
```

Summary

```
summary(mlr)
```

```
##
## Call:
## lm(formula = medv ~ ., data = train.data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.40013 -0.26214 -0.06769  0.17814  2.35738
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.035423   0.023695  -1.495 0.135789
## crim        -0.042440   0.031485  -1.348 0.178506
## zn           0.058543   0.035236   1.661 0.097474 .
## indus       -0.044564   0.046408  -0.960 0.337545
## chas         0.048801   0.026021   1.875 0.061518 .
## nox         -0.181812   0.050075  -3.631 0.000323 ***
## rm          0.327836   0.032518  10.082 < 2e-16 ***
## age        -0.009319   0.042395  -0.220 0.826144
## dis        -0.259040   0.045304  -5.718 2.23e-08 ***
## tax        -0.031723   0.043656  -0.727 0.467900
## ptratio    -0.197012   0.030720  -6.413 4.38e-10 ***
## black       0.074621   0.026127   2.856 0.004533 **
## lstat      -0.324002   0.044322  -7.310 1.68e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4609 on 368 degrees of freedom
## Multiple R-squared:  0.7626, Adjusted R-squared:  0.7549
## F-statistic: 98.52 on 12 and 368 DF,  p-value: < 2.2e-16
```

Predict on Test Data

```
pred_mlr <- predict(mlr, test.data)
```

Descalce

```
#compute mean and sd
mean <- mean(house$medv)
sd <- sd(house$medv)

# compute y(unscaled)
y <- test.data$medv*sd+mean

#compute y^hat (unscaled)
yhat <- pred_mlr*sd+mean
```

Compute Test MSE

```
(MSE.nn.3 = RMSE(y,yhat)^2)
```

```
## [1] 0.507237
```

or

```
sum((y-yhat)^2)/length(y)
```

```
## [1] 0.507237
```